What Is Consistent Hashing?

Last updated: March 18, 2024

Written by: Narendra Kangralkar
(https://www.baeldung.com/cs/author/narendrakangralkar)

Reviewed by: Grzegorz Piwowarek
(https://www.baeldung.com/cs/editor/grzegorz-author)

1. Overview

Nowadays, distributed systems (/cs/distributed-systems-guide) are ubiquitous. We can see its wide adoption in Database Systems, Message Queues (/pub-sub-vs-message-queues)0, Content Delivery Networks (CDN) (/cs/cdn#:~:text=A%20content%20distribution%20network%20or,to%20provide%20faster%20content%20delivery.), etc.

A distributed system consists of multiple components located on different machines that communicate and coordinate actions to appear as a single unit to the end user. One of the major goals of the distributed system is to support horizontal scalability and elasticity.

To achieve this, the distributed system must allow the addition or removal of the nodes from the cluster, and consistent hashing is an ideal solution for such use cases.

In this tutorial, we'll discuss distributed hashing, its limitations, and how consistent hashing allows us to overcome these limitations.

2. What Is Hashing?

Hashing (/cs/hashing#:~:text=Hashing%20is%20a%20one%2Ddirectional,hashes%20are%20160%20bits%20long.) is the process of generating a fixed-length output from a variable-length input. Oftentimes, this fixed-length output is referred to as a *hash value*, *digest*, or *code*. The process that performs this conversion is called a *hash function*.

Hashing is widely used in various data structures. A hash table is a data structure that uses the hashing technique for efficient searching. **This simple hashing mechanism is suitable only if the data set is small enough to fit on a**

single machine. However, this is not true in most cases.

Apart from this, a simple hashing technique doesn't provide high availability in case of failure and thus becomes a single point of failure. In such cases, we can use distributed hashing to overcome some limitations.

3. What Is Distributed Hashing?

In the previous section, we discussed simple hashing and its limitation. Now, let's understand distributed hashing.

Distributed hashing allows us to implement the hash table across multiple machines. **A Distributed Hash Table(DHT) is a decentralized data store that allows us to store and retrieve data efficiently**. The decentralized nature of the Distributed Hash Table allows all nodes to form the collective system without any centralized coordination.

Distributed Hash Tables characteristically emphasize the following properties:

- **Decentralization**: All the nodes of the system collectively form the system without any central coordination
- **Fault Tolerant**: The system is reliable(in some sense), with lots of nodes joining, leaving, and failing at all times
- **Scalable**: The system works efficiently with a large number of nodes

3.1. Limitations of the Distributed Hashing

At first sight, it seems that distributed hashing is a silver bullet as it's fault-tolerant and scalable. However, that's not completely true. Because **the scalability in the distributed hash table is static**.

To understand this, let's see how to use distributed hashing to store the eight keys across the four servers.

First, we calculate the hash of the key, and then we module it with the number of servers. This gives us the server value to store the object associated with a particular key. We can display the same details in a tabular format:

Key	Hash	Hash\%4
key-1	41873	1
key-2	41873	0
key-3	43023	3
key-4	32701	1
key-5	39292	0
key-6	48427	3
key-7	26620	0
key-8	14118	2

In the above table, the last column represents the server number. For example, the *0* value represents the *first server*, the *1* value represents the *second server*, and so on.

So far, we are good, but the real problem starts when there is a change in the number of servers. This can happen when we remove an existing or add a new server to the system.

In such scenarios, we need to redistribute a large amount of the data, and this is where consistent hashing comes into the picture.

4. What Is Consistent Hashing?

Consistent hashing is a technique that works independently of the number of servers. **One of the main goals of consistent hashing is to reduce data redistribution**.

4.1. How Does Consistent Hashing Work?

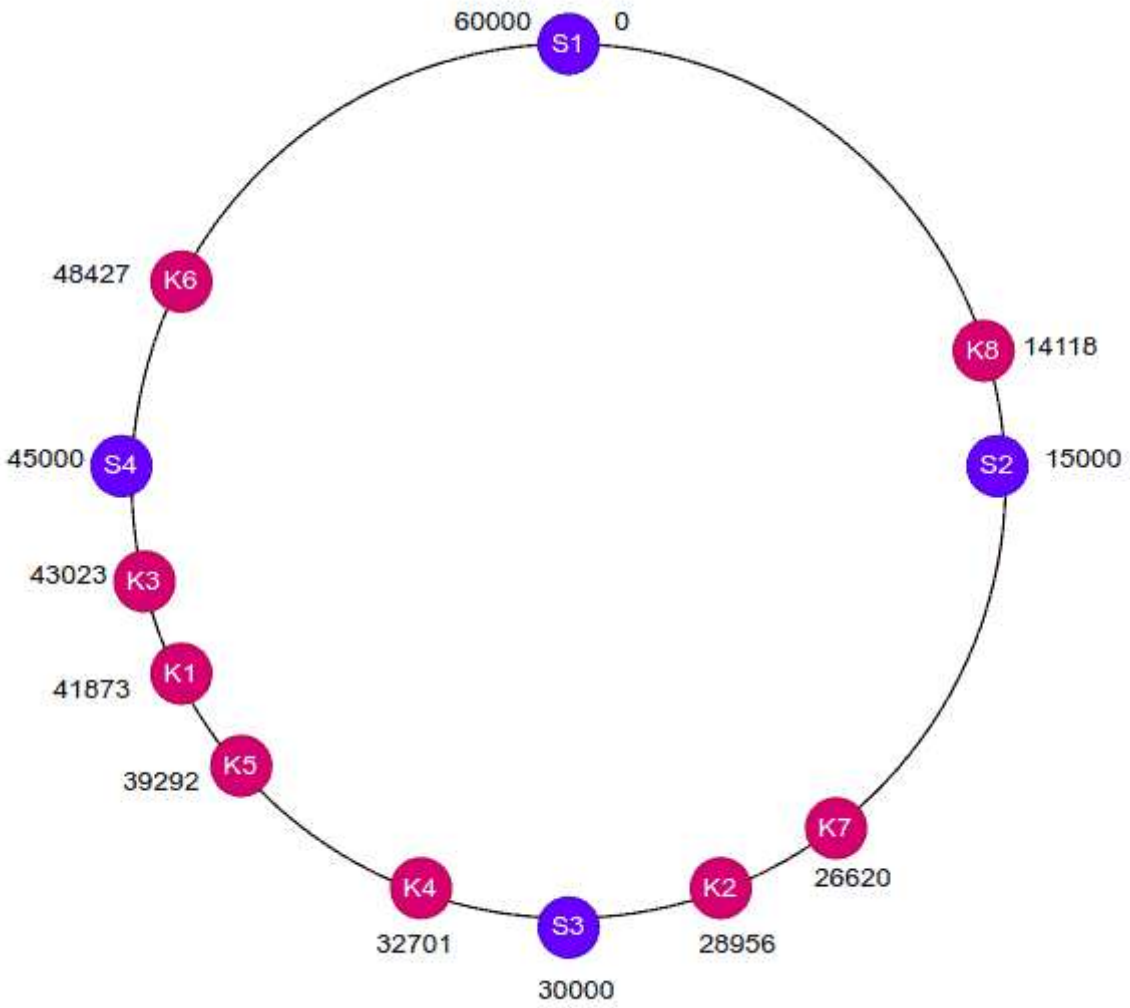
Let's discuss the high-level steps involved in consistent hashing:

1. First, we decide the output range of a hash function. For example, *2**32* or *INT_MAX* or any other value. This range is called *hash space*

2. Then we map this hash space on a logical circle called the *hash ring*
3. Next, we hash all the servers using a hash function and map them on the hash ring
4. Similarly, we hash all the keys using the same hash function and map them on the same hash ring
5. Finally, we traverse in the clockwise direction to locate a server

4.2. Consistent Hashing in Action

To understand this in a better way, let's consider the hash range from *0* to *60,000* and map all the servers and keys on the hash ring:



Now, to locate a server for a particular object, we traverse in a clockwise direction from the current position. The traversal stops when we find the required server.

For example, key *K8* is placed on server *S2*, key *K7* is located on server *S3*, and so on. The below table depicts the same information:

Key	Hash	Server
key-1	41873	server-4
key-2	41873	server-3
key-3	43023	server-4
key-4	32701	server-4
key-5	39292	server-4
key-6	48427	server-1
key-7	26620	server-3
key-8	14118	server-2

As we can see, consistent hashing is independent of the number of servers in the cluster. Hence only a small number of objects need to be redistributed when there is a change in the number of servers.

For example, if we remove server *S2*, we just need to redistribute the key *K8* to the next server, which comes in a clockwise direction. Similarly, if we add a new server between the servers *S4* and *S1* then we just need to redistribute the key *K6* to the newly added server.

In this way, consistent hashing allows elasticity in the cluster with minimal object redistribution.

5. Use Cases of Consistent Hashing

Many well-known distributed systems use consistent hashing. For example, Apache Cassandra (/cassandra-with-java) uses consistent hashing to distribute and replicate data efficiently across the cluster.

Similarly, the Content Delivery Networks(CDN) distribute contents evenly across the edge servers using consistent hashing.

In addition to this, Load Balancers use consistent hashing to distribute persistent connections across backend servers.

6. Conclusion

In this article, we learned about consistent hashing.

First, we discussed simple hashing. Next, we discussed distributed hashing and its limitations. Then, we discussed how consistent hashing helps us to address the limitations of distributed hashing.

Finally, we discussed the practical use cases of consistent hashing.

CATEGORIES

ALGORITHMS (/CS/CATEGORY/ALGORITHMS)
ARTIFICIAL INTELLIGENCE (/CS/CATEGORY/AI)
CORE CONCEPTS (/CS/CATEGORY/CORE-CONCEPTS)
DATA STRUCTURES (/CS/CATEGORY/DATA-STRUCTURES)
LATEX (/CS/CATEGORY/LATEX)
NETWORKING (/CS/CATEGORY/NETWORKING)
SECURITY (/CS/CATEGORY/SECURITY)

SERIES

GRAPHS TUTORIAL (HTTPS://WWW.BAELDUNG.COM/CS/GRAPHS-SERIES)
NEURAL NETWORKS SERIES (HTTPS://WWW.BAELDUNG.COM/CS/NEURAL-NETWORKS-SERIES)
LATEX SERIES (HTTPS://WWW.BAELDUNG.COM/CS/LATEX-SERIES)

ABOUT

ABOUT BAELDUNG (/ABOUT)
BAELDUNG ALL ACCESS (/COURSES)
THE FULL ARCHIVE (/CS/FULL_ARCHIVE)
EDITORS (/EDITORS)

OUR PARTNERS (/PARTNERS/)
PARTNER WITH BAELDUNG (/PARTNERS/WORK-WITH-US)
EBOOKS (/LIBRARY/)
FAQ (/LIBRARY/FAQ)
BAELDUNG PRO (/MEMBERS/)

TERMS OF SERVICE (/TERMS-OF-SERVICE)
PRIVACY POLICY (/PRIVACY-POLICY)
COMPANY INFO (/BAELDUNG-COMPANY-INFO)
CONTACT (/CONTACT)

PRIVACY MANAGER

(/cs/)
(/bael-search)